

Klassifikation handgeschriebener Zeichen mit einem modularen CNN auf dem EMNIST-Datensatz

Modulararbeit SoSe Machine Learning 2

Tsvetan Stanchev

0) Inhaltsverzeichnis

1. Einleitung

2. Aufbau und Struktur des Systems

3. Datensatzvorbereitung

4. Basisklassifikator (TM1)

A) Ziel

B) Wahl des Netzwerks

C) Trainingsprozess

D) Hyperparameteroptimierung

E) Ergebnisse und Evaluation

5. Superklassenmodul (TM2)

A) Ziel

B) Architektur

C) Hyperparameteroptimierung

D) Ergebnisse und Evaluation

6. Modularer Klassifikator

A) Ziel

B) Hyperparameteroptimierung

C) Ergebnisse und Evaluation

7. Fazit

8. Quellenverzeichnis

1) Einleitung

Das Ziel dieses Projekts im Rahmen der Veranstaltung Machine Learning 2 / Deep Learning ist die Entwicklung eines robusten Klassifikators zur Erkennung von handgeschriebenen Zeichen aus dem EMNIST-Datensatz. Der EMNIST-Datensatz stellt eine Erweiterung des bekannten MNIST-Datensatzes dar und umfasst neben Ziffern auch Groß- und Kleinbuchstaben, wodurch sich neue Herausforderungen für die Klassifikation ergeben.

In dieser Projektarbeit wird ein klassischer Deep-Learning-Ansatz auf Basis von Convolutional Neural Networks (CNNs) verfolgt. Zunächst wird ein Basisklassifikator mithilfe von Transfer Learning auf Grundlage des vortrainierten ResNet18-Modells implementiert, der 36 Klassen (Ziffern, A–M, a–m) unterscheidet.

Um eine gleichverteilte Trainingsgrundlage zu gewährleisten, wird ein ausbalancierter Datensatz mit jeweils 5000 Trainings- und 1000 Testbeispielen pro Klasse erstellt. Dabei kommen gezielte Datenaugmentierungen zum Einsatz, um die Klassenverteilung bei unzureichender Originaldatenmenge auszugleichen.

Nach dem Training und der Hyperparameteroptimierung dieses Basisklassifikators wird im zweiten Schritt ein modularer Klassifikator implementiert. Dieser besteht aus zwei Teilmodulen (TM1 und TM2): TM1 klassifiziert wie zuvor die 36 Klassen, während TM2 unabhängig davon die Superklasse (Ziffer, Groß- oder Kleinbuchstabe) eines Zeichens erkennt. Die Ausgaben beider Module werden miteinander kombiniert, um die finale Klassenvorhersage zu verbessern.

Zur Auswahl geeigneter Hyperparameter wird systematisch Optuna eingesetzt. Die gesamte Trainings- und Auswertungsinfrastruktur ist modular aufgebaut, sodass die Modelle leicht auf andere Architekturen oder Aufgaben erweitert werden können. Die Ergebnisse bei der Klassifikatoren werden miteinander verglichen und durch Konfusionsmatrizen visualisiert.

2) Aufbau und Struktur des Systems

Das Projekt ist modular aufgebaut und in mehrere logisch getrennte Komponenten gegliedert, die in der folgenden Abbildung als UML-Diagramm dargestellt sind. Die Gesamtarchitektur umfasst die Bereiche Datenverarbeitung, Modellierung, Training und Optimierung, Evaluation sowie das Hauptskript, das alle Schritte zusammenführt.

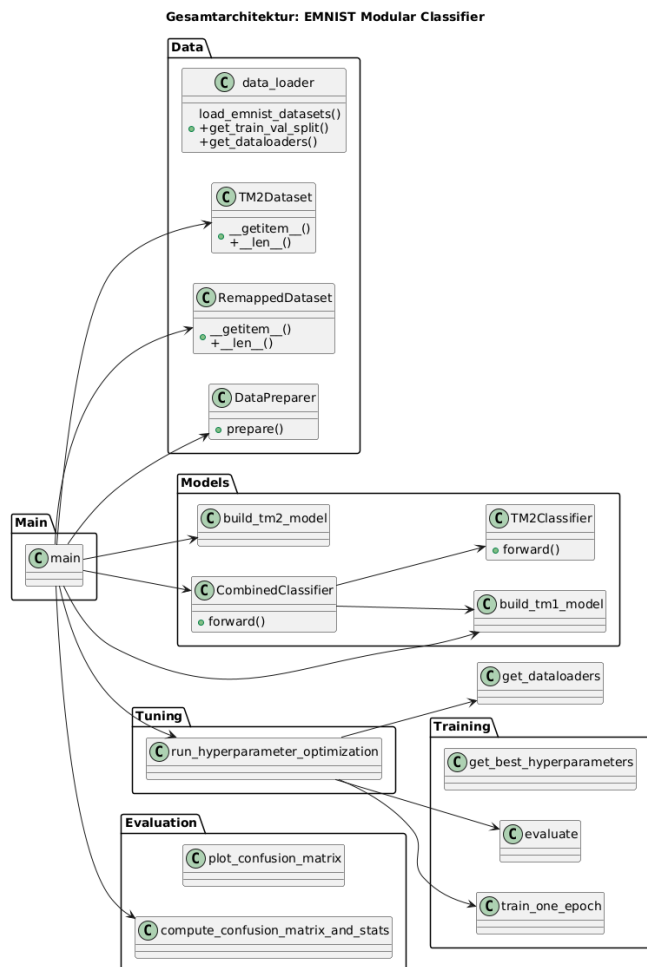
Die Datenvorbereitung erfolgt über die Klasse DataPreparer, welche aus dem EMNIST-Originaldatensatz einen ausbalancierten Trainings- und Testdatensatz mit jeweils genau 5000 Trainings- und 1000 Testbeispielen pro Klasse erstellt. Zur Labelverarbeitung werden zwei benutzerdefinierte Dataset-Klassen eingesetzt: RemappedDataset zur Reduktion auf 36 Zielklassen (Ziffern, A–M, a–m) und TM2Dataset zur Gruppierung in drei Superklassen (Ziffer, Großbuchstabe, Kleinbuchstabe). Die Funktion „get_data loaders“ übernimmt die Aufteilung und Bereitstellung der Daten für Training, Validierung und Test mithilfe von PyTorch-Dataloaders.

Die Modellierung erfolgt in zwei Stufen. Das erste Teilmodul TM1 wird über die Funktion „build_tm1_model“ realisiert und basiert auf einem vortrainierten ResNet18-Modell, das für den Einsatz mit 1-kanaligen EMNIST-Bildern angepasst wurde. Für TM2 wird ein eigenes CNN-Modell (TM2Classifier) entworfen, das ohne Transfer Learning arbeitet und die jeweilige Superklasse vorhersagt. Beide Module werden im CombinedClassifier zusammengeführt, wobei die Vorhersagen von TM1 mit den Wahrscheinlichkeiten von TM2 multipliziert werden, um die finale Klassenvorhersage zu verbessern. Dieses kombinierte Modell wird vollständig end-to-end trainiert.

Das Training und die Hyperparameteroptimierung werden über die Funktionen „train_one_epoch“ und „run_hyperparameter_optimization“ gesteuert. Dabei wird Optuna verwendet, um die wichtigsten Hyperparameter zu optimieren. Als Optimierungsstrategie wird Tree-structured Parzen Estimator (TPE) mit Successive Halving Pruning eingesetzt, um Rechenzeit zu sparen und schwache Konfigurationen frühzeitig zu verwerfen.

Die Auswertung erfolgt durch das Modul „evaluation.py“, welches Funktionen zur Berechnung und Visualisierung von Konfusionsmatrizen bereitstellt. Neben der Gesamtgenauigkeit werden auch Klassengenauigkeiten und Fehlermuster ausgewertet.

Das Hauptskript „main.py“ orchestriert alle Komponenten: Es bereitet die Daten vor, startet die Hyperparameteroptimierung für TM1, TM2 und das kombinierte Modell, und führt anschließend die finale Evaluation durch. Dadurch ergibt sich eine durchgängige



Pipeline von der Datenerstellung bis zur Modellbewertung, die vollständig reproduzierbar und modular erweiterbar ist.

3) Datensatzvorbereitung

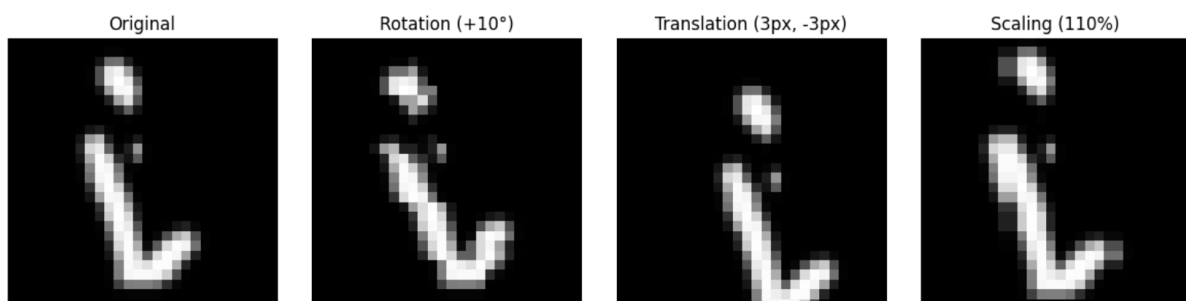
Für das Training der Klassifikationsmodelle wurde der EMNIST-Datensatz in der Variante ByClass verwendet. Da die ursprüngliche Datenverteilung stark unausgeglichen ist und nicht für alle Klassen ausreichend viele Beispiele vorhanden sind, wurde ein automatisierter Vorverarbeitungsschritt implementiert, der die Erzeugung eines ausbalancierten Datensatzes ermöglicht.

Ziel dieser Klasse ist es, sowohl für das Training als auch für den Testsatz exakt 5000 Trainings- und 1000 Testbeispiele pro Klasse zu erzeugen. Die Zielklassen sind fest in der Klasse definiert. Der Rohdatensatz wird geladen und zunächst mit der auf eine möglichst gleichmäßige Verteilung reduziert. Ist die Anzahl an Beispielen für eine Klasse kleiner als erforderlich, werden zusätzliche Daten durch gezielte Augmentierung synthetisch erzeugt.

Die Datenaugmentation erfolgt durch zufällige affine Transformationen. Sie erhöht so die Varianz innerhalb einer Klasse künstlich. Es werden dabei gezielt Rotation, Translation und Skalierung angewendet:

- Rotation: Die Bilder werden zufällig um bis zu ± 10 Grad gedreht. Dies simuliert typische Schwankungen in der Handschrift, z. B. leicht geneigte Ziffern oder Buchstaben.
- Translation: Die Bildinhalte werden zufällig in horizontaler und vertikaler Richtung verschoben – jeweils bis zu 10 % der Bildgröße.
- Skalierung: Zusätzlich wird das Bild skaliert, d. h. in der Größe verändert – zufällig zwischen 90 % und 110 % der Originalgröße.

Die gewählten Transformationsarten – Rotation, Translation und Skalierung – sind dabei sinnvoll gewählt und realitätsnah, sodass sie die semantische Bedeutung der Zeichen nicht verändern und die Integrität der Daten erhalten bleibt.



Nach der Erzeugung aller Beispiele werden die Daten in Tensor-Form überführt und die Labels in einen zusammenhängenden Labelbereich von 0 bis 35 remapped, um sie für das Training nutzbar zu machen. Die train- und test-Samples werden anschließend als

TensorDataset-Objekte zurückgegeben und für das Training bzw. die Evaluation der Modelle gespeichert.

Dieser Schritt stellt sicher, dass alle folgenden Modelle auf einem vollständig balancierten Datensatz trainiert und evaluiert werden.

4) Basisklassifikator (TM1)

A) Ziel

Der erste Teil des Projekts verfolgt das Ziel, mithilfe von Transfer Learning ein Convolutional Neural Network (CNN) zu trainieren, das die Klassifikation der Eingabebilder in vordefinierte Klassen ermöglicht. Statt ein neuronales Netz vollständig neu zu trainieren, wird ein bereits vortrainiertes ResNet18-Modell verwendet, das ursprünglich auf dem ImageNet-Datensatz trainiert wurde. Dieses Vorgehen erlaubt es, bereits gelernte Bildmerkmale (wie Kanten, Formen und Texturen) für die neue Klassifikationsaufgabe zu nutzen. Dadurch wird die Trainingszeit verkürzt und die Modellgenauigkeit erhöht.

B) Wahl des Netzwerks

ResNet18 wurde gewählt, da es schneller trainierbar ist als tiefere Netzwerke und dennoch gute Ergebnisse liefert. Das Netz wurde so angepasst, dass es Graustufenbilder verarbeitet und 36 Klassen unterscheidet. Dazu wurde der erste Convolutional Layer auf 1-Kanal-Eingaben angepasst und der finale Fully-Connected-Layer durch eine Kombination aus Dropout-Schicht und einem Linearlayer mit 36 Ausgängen ersetzt.

C) Trainingsprozess

Das Training des Klassifikationsmodells erfolgt ausschließlich im Rahmen der Hyperparameteroptimierung und wird nicht direkt in der Hauptdatei aufgerufen.

Das Training erfolgt mittels einer klassischen supervised-learning-Pipeline: Als Verlustfunktion wird die Kreuzentropie verwendet, und die Modellparameter werden iterativ mithilfe eines geeigneten Optimierungsverfahrens angepasst. Während des Trainings wird die Leistung regelmäßig auf einem separaten Validierungsdatensatz überwacht, um den Lernfortschritt zu kontrollieren und ein mögliches Überanpassen frühzeitig zu erkennen.

Zur Stabilisierung des Trainingsprozesses kommt ein Early-Stopping-Mechanismus zum Einsatz, der das Training automatisch abbricht, wenn sich die Validierungsgenauigkeit über mehrere Epochen hinweg nicht verbessert. Zusätzlich wird die Modellversion mit der besten Validierungsleistung gespeichert, um die bestmögliche Konfiguration für die spätere Evaluation bereitzustellen.

Das eigentliche Training findet innerhalb einer Trainingsfunktion statt, die im Rahmen der Hyperparameteroptimierung von Optuna mehrfach aufgerufen wird. Jede

Konfiguration durchläuft dabei denselben Trainings- und Validierungsprozess, sodass eine faire und systematische Bewertung gewährleistet ist.

D) Hyperparameteroptimierung

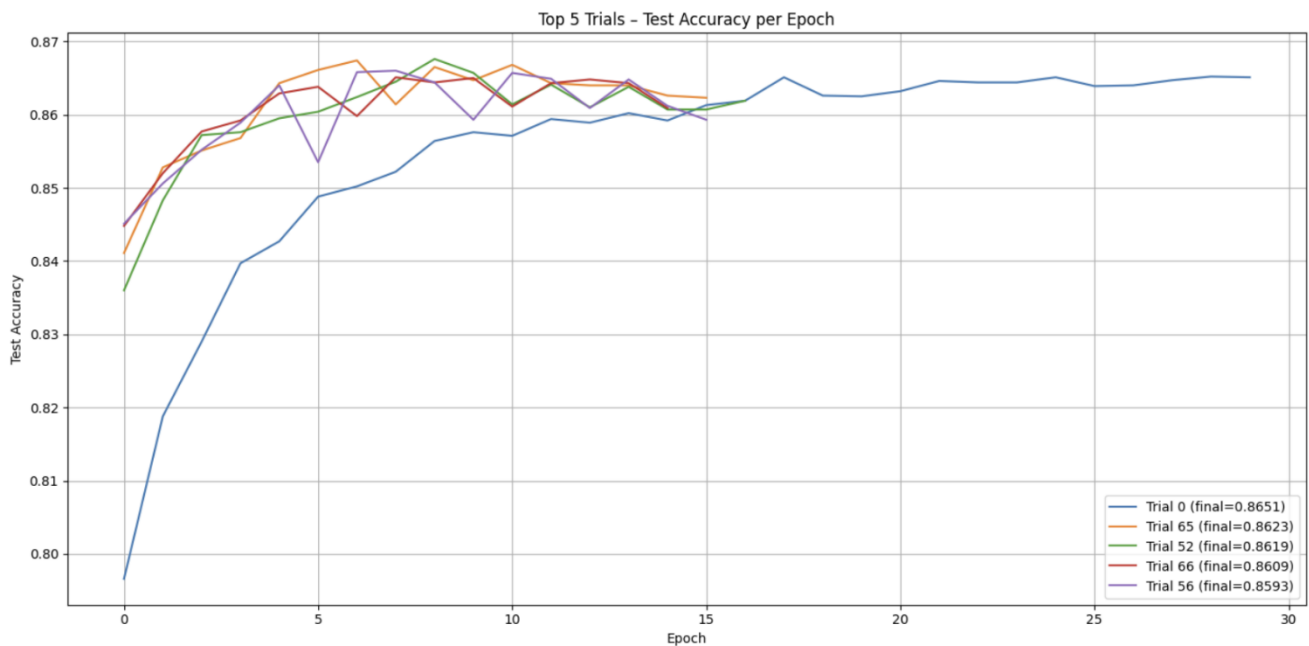
Um die bestmögliche Modellleistung zu erzielen, werden zentrale Trainingsparameter systematisch durch Hyperparameteroptimierung angepasst. Diese Optimierung ist entscheidend, da viele Hyperparameter einen starken Einfluss auf die Konvergenz und Generalisierungsfähigkeit des Modells haben. Eine manuelle Abstimmung wäre ineffizient, zeitaufwendig und nicht reproduzierbar. Ziel ist daher, durch automatisierte Optimierung eine robuste Konfiguration zu finden, die eine hohe Validierungsgenauigkeit gewährleistet.

Für die Optimierung kommt das Framework Optuna zum Einsatz. Optuna erlaubt eine flexible Definition des Suchraums sowie eine objektive Bewertung durch eine sogenannte `objective()`-Funktion. Besonders vorteilhaft ist die Unterstützung für Pruning-Strategien und Early-Stopping.

Im Rahmen der Optimierung werden spezifische Hyperparameter untersucht:

- Lernrate (lr): Die Lernrate bestimmt die Schrittgröße, mit der das Modell seine Gewichte während des Trainings anpasst. Ist sie zu hoch, kann das Modell divergieren; ist sie zu niedrig, konvergiert es nur sehr langsam. Eine optimale Lernrate ist entscheidend für ein stabiles und effizientes Training.
- Dropout (dropout): Dropout hilft, Überanpassung (Overfitting) zu vermeiden, indem es zufällig Neuronen während des Trainings deaktiviert. Der ideale Wert hängt von der Modellkomplexität und der Datengröße ab. Ein zu hoher Wert kann das Lernen behindern, ein zu niedriger führt zu schlechterer Generalisierung.
- Batchgröße (batch_size): Die Batchgröße beeinflusst die Trainingsdynamik und den Speicherverbrauch. Kleinere Batches führen zu stabilerer Generalisierung, benötigen aber mehr Zeit pro Epoche. Größere Batches beschleunigen das Training, können aber weniger robust gegen lokale Minima sein.
- Optimierer (optimizer): Der gewählte Optimierungsalgorithmus steuert, wie die Gewichte aktualisiert werden. SGD (mit Momentum) ist bewährt, benötigt aber oft mehr Feintuning. AdamW kombiniert adaptives Lernen mit Weight Decay und liefert oft gute Resultate ohne viele Anpassungen.
- Weight Decay (weight_decay): Dieser Regularisierungsterm verhindert, dass Gewichte zu groß werden, und reduziert damit das Risiko von Overfitting. Der optimale Wert hängt stark von der Datenkomplexität ab – zu viel Regularisierung kann aber die Modellkapazität einschränken.

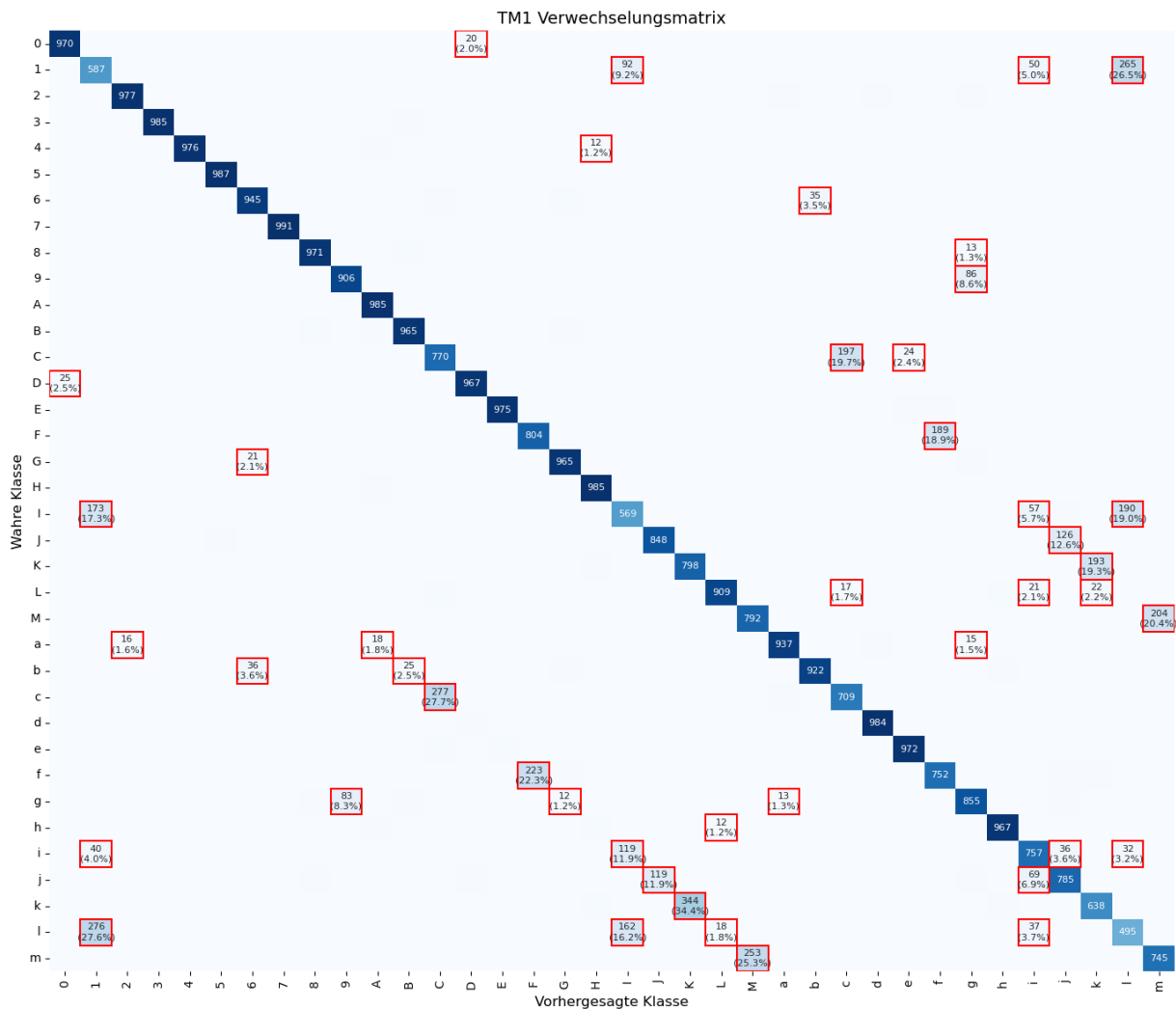
E) Ergebnisse und Evaluation



Nach der Hyperparameteroptimierung wurde das beste TM1-Modell basierend auf der finalen Testgenauigkeit ausgewählt und evaluiert. Die Abbildung zeigt den Verlauf der Testgenauigkeit über 30 Epochen für die fünf besten Konfigurationen.

Man erkennt, dass alle Top-Trials sehr schnell – oft bereits nach 5 – 8 Epochen – Testgenauigkeiten von über 85 % erreichen. Besonders Trial 0 hebt sich ab: Trotz eines langsameren Anfangs erreicht es mit 0.8651 die höchste finale Testgenauigkeit. Die anderen Top-Trials (z. B. Trial 55, 52 und 66) liegen eng beieinander, mit Endwerten zwischen 0.8593 und 0.8623.

Insgesamt bestätigen diese Ergebnisse die hohe Robustheit des gewählten ResNet18-Modells in Kombination mit gezielter Hyperparameteroptimierung. Die erzielten Testgenauigkeiten belegen die gute Generalisierungsfähigkeit des Modells auf den zuvor nicht gesehenen EMNIST-Testdaten.



Die abgebildete Verwechslungsmatrix visualisiert die Vorhersagen des TM1-Klassifikators auf dem Testdatensatz. Jeder Eintrag zeigt die Anzahl der Beispiele einer tatsächlichen Klasse (Zeile), die als eine bestimmte Klasse (Spalte) vorhergesagt wurden. Korrekte Vorhersagen liegen auf der Diagonalen, während Fehler außerhalb der Diagonale markiert sind – größere Verwechslungen ($> 1\%$) sind rot hervorgehoben.

Die Mehrheit der Klassen wird mit sehr hoher Genauigkeit erkannt – viele Diagonaleinträge liegen bei 970–990 korrekt klassifizierten Beispielen pro Klasse. Besonders robuste Vorhersagen zeigen sich bei Ziffern wie 0, 2, 4, 6, 7, 8 sowie Buchstaben wie C, I, L, N.

Beim genaueren Blick auf die Verwechslungsmatrix des modularen Klassifikators lassen sich mehrere charakteristische Fehlmuster erkennen. Besonders auffällig ist die häufige Verwechslung der Ziffer „1“ mit den Buchstaben „l“ (kleines L, 92-mal) und „I“ (großes i, 50-mal), was auf deren visuelle Ähnlichkeit zurückzuführen ist.

Im Bereich der Kleinbuchstaben treten mehrere systematische Fehler auf: So wird „h“ häufig mit „k“ (276-mal, entsprechend 27,6 %) verwechselt, „m“ mit „n“ (253-mal, 25,3 %) und „f“ mit „t“ (83-mal, 8,3 %). Auch „c“ wird oft als „e“ (197-mal, 19,7 %) und

„d“ als „a“ (173-mal, 17,3 %) klassifiziert. Diese Verwechslungen deuten auf strukturelle Ähnlichkeiten in den handschriftlichen Darstellungen hin.

Ein weiteres zentrales Muster betrifft die Unterscheidung zwischen Groß- und Kleinbuchstaben: So wird beispielsweise „A“ in 277 Fällen (27,7 %) als „a“, „E“ in 204 Fällen (20,4 %) als „e“ und „M“ in 162 Fällen (16,2 %) als „m“ erkannt. Diese Fehler unterstreichen die Herausforderung, Groß- und Kleinbuchstaben zuverlässig zu trennen – insbesondere bei ähnlicher Handschrift und Formgebung.

Die meisten Fehler sind auf visuelle Ähnlichkeiten im EMNIST-Zeichensatz zurückzuführen und wären auch für Menschen schwer zu unterscheiden.

5) Superklassenmodul (TM2)

A) Ziel

Das Ziel des TM2-Moduls besteht darin, jedes Eingabebild nicht in eine von 36 Klassen, sondern in eine von drei Superklassen einzuordnen: Ziffer, Großbuchstabe oder Kleinbuchstabe. Diese Einteilung bildet eine zusätzliche strukturelle Information, die unabhängig vom Hauptklassifikator (TM1) gelernt wird. Durch die separate Vorhersage der Zeichenkategorie soll die spätere Klassifikation gezielter erfolgen, da beispielsweise ein Zeichen, das klar als Ziffer erkannt wird, nicht mehr als Buchstabe fehlerklassifiziert werden sollte. TM2 dient somit als Hilfsmodul, das die finale Entscheidung im kombinierten Klassifikator unterstützt und insbesondere dazu beiträgt, Verwechslungen zwischen strukturell unterschiedlichen Klassen (z. B. „a“ vs. „1“) zu reduzieren.

B) Architektur

Das TM2-Modul wurde als eigenständiges Convolutional Neural Network (CNN) entworfen, das speziell auf die Superklassenklassifikation (Ziffer, Großbuchstabe, Kleinbuchstabe) ausgerichtet ist. Im Gegensatz zum TM1-Modell, das auf Transfer Learning basiert, wird TM2 komplett eigenständig trainiert und verwendet eine schlankere, flexibel konfigurierbare Architektur, die gezielt auf die reduzierte Anzahl an Zielklassen abgestimmt ist.

Der Netzwerkaufbau besteht aus mehreren Convolutional-Blöcken, jeweils gefolgt von Aktivierungs- und Pooling-Schritten. Die Anzahl der Blöcke sowie die Anzahl der Faltungsschichten pro Block werden dabei als Hyperparameter optimiert. Im Anschluss an die Faltungsstufen wird der Feature-Map-Ausgang über ein oder mehrere voll verbundene Schichten (Fully Connected Layers) klassifiziert.

Ein Dropout-Layer zwischen den linearen Schichten dient zur Regularisierung, um Überanpassung zu vermeiden. Der finale Ausgabelayer liefert eine Wahrscheinlichkeitsverteilung über die drei Superklassen.

Durch die modulare Struktur ist es möglich, die Architektur an die Komplexität der Aufgabe und die Datenmenge anzupassen, was insbesondere in der Hyperparameteroptimierung gezielt ausgenutzt wird.

C) Hyperparameteroptimierung

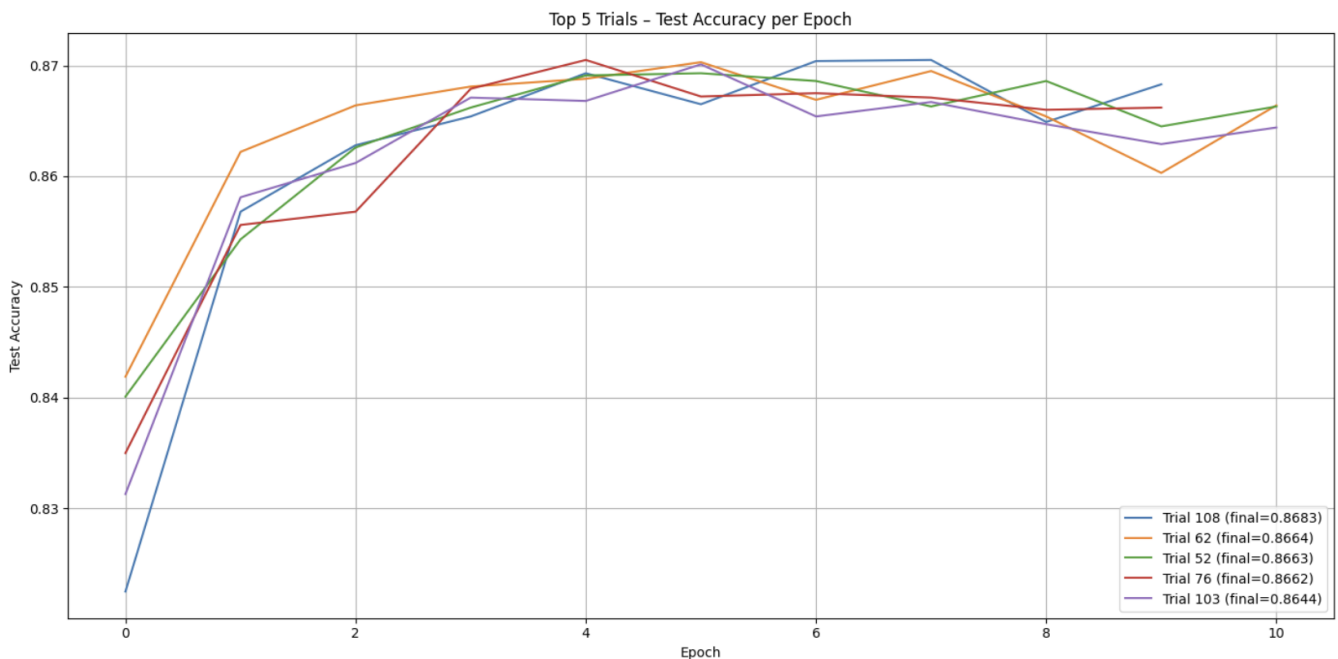
Für das TM2-Modul wird eine gezielte Hyperparameteroptimierung durchgeführt. Da es sich bei TM2 um ein eigenständig definiertes Convolutional Neural Network handelt, ist die Architektur vollständig konfigurierbar. Um ein möglichst leistungsfähiges Modell zu erhalten, werden sowohl trainingsbezogene als auch architekturspezifische Hyperparameter untersucht.

Zunächst werden dieselben fünf trainingsbezogenen Hyperparameter wie bei TM1 optimiert: Lernrate, Dropout, Batchgröße, Optimierer und Weight Decay. Diese beeinflussen maßgeblich die Trainingsdynamik und das Generalisierungsverhalten des Modells. Zusätzlich werden folgende architekturspezifische Hyperparameter untersucht:

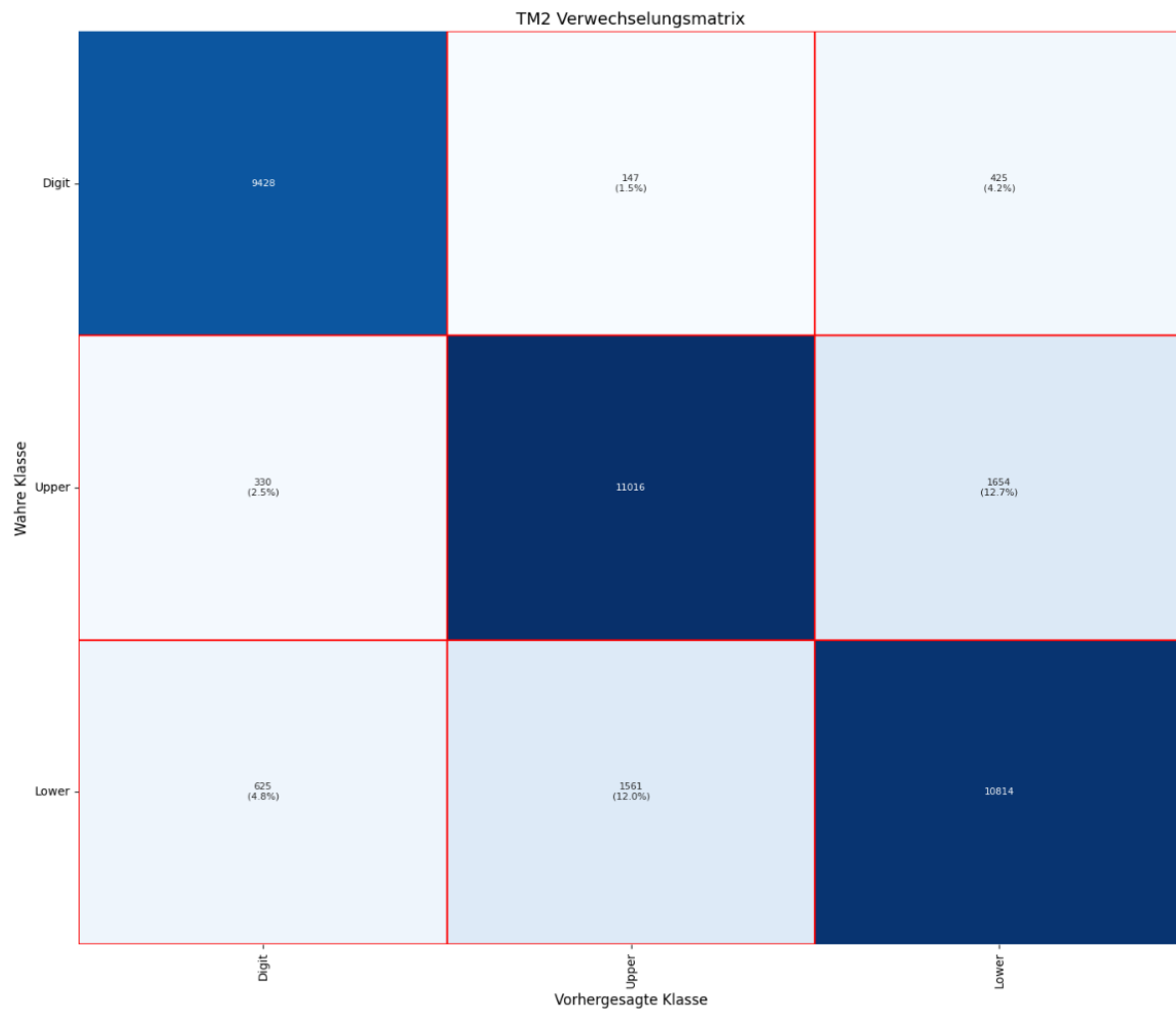
- Anzahl der Convolutional-Blöcke (num_blocks): Bestimmt die Tiefe des Netzwerks. Mehr Blöcke ermöglichen komplexere Repräsentationen, erhöhen jedoch auch die Rechenkosten.
- Anzahl der Schichten pro Block (layers_per_block): Je mehr Schichten, desto höher die Ausdrucksstärke – allerdings steigt damit auch die Gefahr der Überanpassung.
- Größe der versteckten Schicht (hidden_dim): Beeinflusst die Kapazität der Klassifikationsschicht.
- Basisanzahl der Kanäle (base_channels): Definiert die anfängliche Breite der Convolutional-Schichten. Die Kanalanzahl verdoppelt sich mit jedem Block.
- Skip Connections (use_skip): Optional aktivierbar. Sie können das Training tieferer Netze erleichtern, indem sie den Gradientenfluss verbessern.

Die letzten fünf architekturspezifischen Hyperparameter von TM2 werden nicht erneut im Combined-Modul optimiert, sondern direkt aus der besten TM2-Konfiguration übernommen. Dadurch bleibt die Struktur von TM2 im kombinierten Training konsistent und der Fokus der Optimierung kann auf die gemeinsamen Trainingsparameter gelegt werden. Dies reduziert die Komplexität der Optimierung und verhindert inkonsistente Architekturanpassungen.

D) Ergebnisse und Evaluation



Die Abbildung zeigt die Entwicklung der Testgenauigkeit der fünf besten Trials des Superklassenmoduls TM2 über den Trainingsverlauf. Bereits nach wenigen Epochen überschreiten alle Top-Trials die Marke von 86 %, wobei sich die Kurven ab Epoche 4–5 deutlich stabilisieren. Besonders hervorzuheben ist Trial 108, das mit einer finalen Testgenauigkeit von 0.8683 die beste Leistung erzielt. Die übrigen Top-Konfigurationen (Trials 62, 52, 76 und 103) bewegen sich in einem engen Bereich zwischen 0.8644 und 0.8664, was auf eine konsistente und robuste Optimierung hinweist. Insgesamt zeigt TM2 damit ein stabiles und verlässliches Verhalten und erfüllt seine Rolle als semantischer Superklassen-Vorfilter im modularen Klassifikationssystem sehr zuverlässig.



Die Verwechslungsmatrix des TM2-Moduls zeigt die Klassifikationsergebnisse auf der Ebene der drei Superklassen: Ziffer (Digit), Großbuchstabe (Upper) und Kleinbuchstabe (Lower). Die diagonalen Einträge, die korrekte Vorhersagen darstellen, weisen sehr hohe Werte auf – mit über 94 % für Ziffern, etwa 85 % für Großbuchstaben und rund 83 % für Kleinbuchstaben. Auffällig ist jedoch, dass insbesondere Kleinbuchstaben häufiger mit Großbuchstaben verwechselt werden (12,0 %) und umgekehrt (12,7 %). Dies lässt sich durch die visuelle Ähnlichkeit vieler Buchstaben erklären (z. B. 'c' und 'C' oder 'o' und 'O'), insbesondere im EMNIST-Datensatz, der auf handschriftlichen Eingaben basiert. Auch die Verwechslung von Ziffern mit Kleinbuchstaben ist mit 4,8 % noch relevant. Insgesamt bestätigt die Matrix jedoch die hohe Zuverlässigkeit des Moduls: Die drei Hauptklassen werden überwiegend korrekt erkannt, sodass TM2 seine Funktion als semantische Vorauswahl für die kombinierte Architektur erfolgreich erfüllt.

6) Modularer Klassifikator

A) Ziel

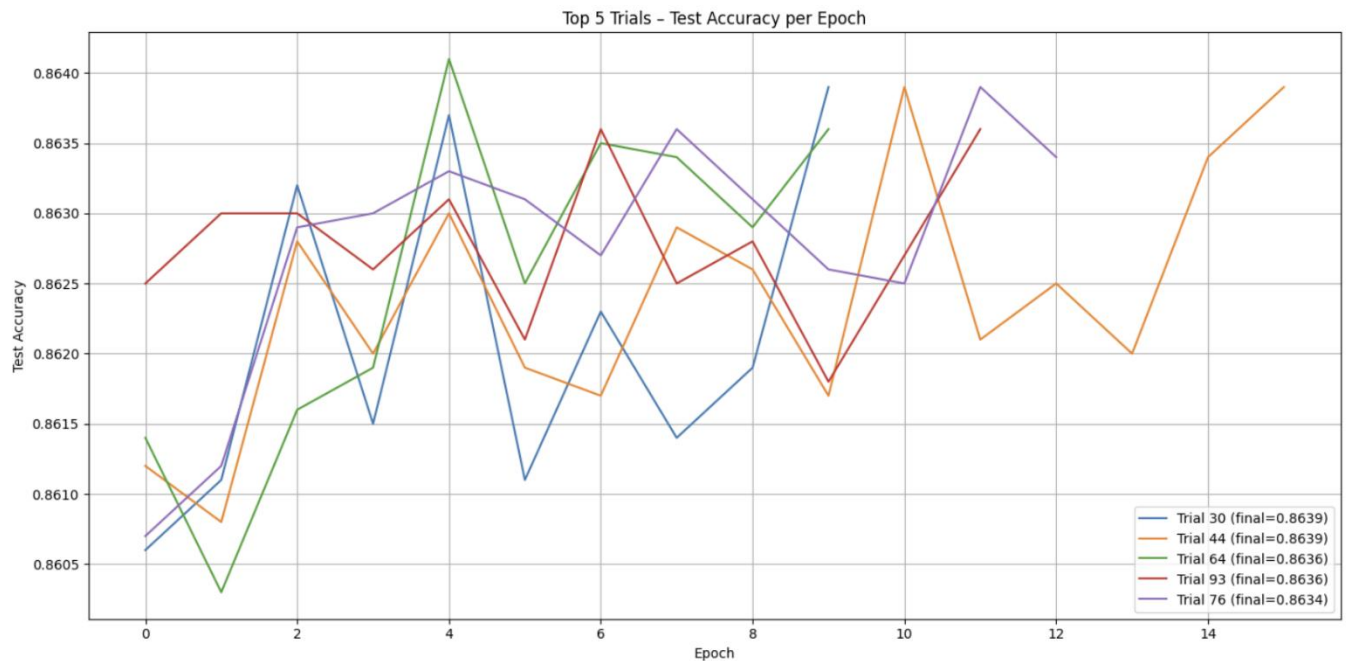
Der modulare Klassifikator kombiniert zwei spezialisierte Teilmodelle: TM1 für die Feinklassifikation in 36 Zielklassen (Ziffern und Buchstaben A–M, a–m) und TM2 für die Grobklassifikation in drei Superklassen (Digit, Upper, Lower). Beide Modelle verarbeiten parallel das gleiche Eingabebild und erzeugen unabhängig voneinander ihre jeweiligen Vorhersagen. Die finale Klassifikation erfolgt durch eine gewichtete Kombination beider Ausgaben: Die TM2-Ausgabe wird in eine Gewichtungsmaske überführt, die jeder der 36 Klassen eine Relevanz basierend auf der Superklassenwahrscheinlichkeit zuweist. Diese Maske wird mit der TM1-Ausgabe multiplikativ verrechnet, sodass beispielsweise Ziffernklassen bei hoher Digit-Wahrscheinlichkeit stärker gewichtet werden. Durch diese Kombination wird das Modell in seiner Entscheidung gezielt unterstützt, insbesondere bei schwer unterscheidbaren Klassen mit ähnlichem Aussehen.

B) Hyperparameteroptimierung

Das Training des modularen Klassifikators erfolgt end-to-end, wobei beide Teilmodelle – TM1 und TM2 – gleichzeitig optimiert werden. Dabei werden die Gewichte beider Module gemeinsam aktualisiert, sodass sie lernen, sich gegenseitig optimal zu ergänzen. Der kombinierte Output ergibt sich aus der Multiplikation der TM1-Klassifikation (36 Klassen) mit einer Gewichtungsmaske, die aus der TM2-Superklassenprognose erzeugt wird. Diese Methode erzwingt eine implizite Zusammenarbeit beider Module: TM2 lernt, die jeweils zutreffende Superklasse zuverlässig zu identifizieren, während TM1 sich auf die Unterscheidung innerhalb dieser Gruppe konzentriert. Die trainingsbezogenen Hyperparameter (wie Lernrate, Batchgröße, Optimierer usw.) werden durch eine eigenständige Optuna-Optimierung für den kombinierten Ansatz gesucht. Im Gegensatz dazu bleiben die architekturspezifischen Parameter von TM2 (z. B. Anzahl der Blöcke, Schichten, Kanäle) während des Trainings des kombinierten Modells konstant und werden direkt aus dem besten TM2-Konfigurationslauf übernommen.

Die Hyperparameteroptimierung des modularen Klassifikators erfolgt erneut mit Optuna unter Verwendung der Successive-Halving-Strategie. Ziel ist es, die trainingsbezogenen Parameter zu finden, die zu einer optimalen gemeinsamen Anpassung beider Teilmodelle führen. Zusätzlich wird durch die parallele Optimierung beider Teilmodelle sichergestellt, dass TM2 seine Gruppierungsfunktion effektiv erfüllt, während TM1 gleichzeitig die feingranulare Klassifikation innerhalb der Superklassen erlernt. Die Bewertung erfolgt anhand der Validierungsgenauigkeit des kombinierten Outputs.

C) Ergebnisse und Evaluation



Die Auswertung des modularen Klassifikators zeigt, dass die besten fünf Trials der Hyperparameteroptimierung Testgenauigkeiten zwischen 86,34 % und 86,39 % erreichen (siehe Abbildung). Diese Werte liegen minimal unterhalb der besten Resultate des TM1-Modells, welches bis zu 86,51 % erzielte. Die leicht geringere Testgenauigkeit lässt sich dadurch erklären, dass das kombinierte Modell nicht nur auf die feinaufgelöste Vorhersage des TM1-Moduls vertraut, sondern zusätzlich durch die Superklassen-Ausgabe von TM2 gewichtet wird. Wenn TM2 beispielsweise einen Buchstaben fälschlicherweise der falschen Superklasse zuordnet, kann dies die finale Ausgabe des Modells verfälschen – etwa durch eine ungewollte Verstärkung oder Abschwächung der TM1-Wahrscheinlichkeiten. Dadurch entsteht ein zusätzlicher Unsicherheitsfaktor im Vorhersageprozess. Trotz dieser zusätzlichen Komplexität zeigen die Testgenauigkeitsverläufe der besten Trials stabile Lernkurven, was auf eine gewisse Robustheit und Regularisierungswirkung der TM2-Komponente hindeutet.

Modell insgesamt eine etwas stärkere Diagonale auf. Dies deutet darauf hin, dass die Einbeziehung der Superklasseninformation im modularen Aufbau zwar zusätzliche Struktur liefert, aber in einigen Fällen die feingranulare Trennung einzelner Klassen erschwert.

7) Fazit

Die Evaluierung zeigt, dass der modulare Ansatz vergleichbare Ergebnisse liefert wie das direkte TM1-Modell. Obwohl das kombinierte Modell in seiner Architektur deutlich komplexer ist und auf eine explizite Trennung von Superklassen (TM2) und Feinklassifikation (TM1) setzt, konnte es keine signifikante Leistungssteigerung gegenüber dem Basismodell erzielen. Dies könnte darauf zurückzuführen sein, dass die Multiplikation der TM1- und TM2-Ausgaben eine zusätzliche Fehlerquelle darstellt: Schon geringe Unsicherheiten oder Fehleinschätzungen in der Superklassenklassifikation durch TM2 können zu falsch gewichteten Wahrscheinlichkeiten im finalen Output führen und so die Gesamtgenauigkeit negativ beeinflussen. Dennoch zeigt sich, dass das modulare System grundsätzlich in der Lage ist, die intendierte Zusammenarbeit zwischen den beiden Teilmodellen erfolgreich umzusetzen.

8) Quellenverzeichnis

- [1] Cohen, G., Afshar, S., Tapson, J., and van Schaik, A. (2017). EMNIST: an extension of MNIST to handwritten letters, arXiv:1702.05373v2 <http://arxiv.org/abs/1702.05373>
- [2] Optuna Developers: *Optuna: A hyperparameter optimization framework*. <https://optuna.org/> – Zugriff: 27.06.2025.
- [3] Torchvision Documentation – *ResNet18*. <https://pytorch.org/vision/stable/models/generated/torchvision.models.resnet18.html> – Zugriff: 27.06.2025.
- [4] A. Paszke et al.: *PyTorch: An Imperative Style, High-Performance Deep Learning Library*. Advances in Neural Information Processing Systems 32 (NeurIPS 2019).
- [5] matplotlib, pandas, numpy, scikit-learn: *Offizielle Dokumentation und Verwendung in Python-Projekten*. <https://pypi.org/> – Zugriff: 27.06.2025.